Floating Point Representation – Basics

The following description explains terminology and primary details of IEEE 754 binary floating point representation. The discussion confines to single and double precision formats.

Usually, a real number in binary will be represented in the following format,

$$I_m I_{m-1} \ldots I_2 I_1 I_0 . F_1 F_2 \ldots F_n F_{n-1}$$

Where I_m and F_n will be either 0 or 1 of integer and fraction parts respectively.
A finite number can also represented by four integers components, a sign (s), a base (b), a significant (m), and an exponent (e). Then the numerical value of the number is evaluated as

$$(-1)^s \times m \times b^e \text{——} \text{Where } m < |b|$$

Depending on base and the number of bits used to encode various components, the IEEE 754 standard defines five basic formats. Among the five formats, the binary32 and the binary64 formats are single precision and double precision formats respectively in which the base is 2.

Table – 1 Precision Representation

Precision	Base	Sign	Exponent	Significant
Single precision	2	1	8	23+1
Double precision	2	1	11	52+1

Single Precision Format:
As mentioned in Table 1 the single precision format has 23 bits for significant (1 represents implied bit, details below), 8 bits for exponent and 1 bit for sign.

For example, the rational number 9÷2 can be converted to single precision float format as following,

$$9_{(10)} \div 2_{(10)} = 4.5_{(10)} = 100.1_{(2)}$$

The result said to be ***normalized***, if it is represented with leading 1 bit, i.e. $1.001_{(2)} \times 2^2$. (Similarly when the number $0.000000001101_{(2)} \times 2^3$ is normalized, it appears as $1.101_{(2)} \times 2^{-6}$). Omitting this implied 1 on left extreme gives us the ***mantissa*** of float number. A normalized number provides more accuracy than corresponding ***de-***

normalized number. The implied most significant bit can be used to represent even more accurate significant (23 + 1 = 24 bits) which is
called ***subnormal*** representation. *The floating point numbers are to be represented in normalized form*.
The subnormal numbers fall into the category of de-normalized numbers. The subnormal representation slightly reduces the exponent range and can't be normalized since that would result in an exponent which doesn't fit in the field. Subnormal numbers are less accurate, i.e. they have less room for nonzero bits in the fraction field, than normalized numbers. Indeed, the accuracy drops as the size of the subnormal number decreases. However, the subnormal representation is useful in filing gaps of floating point scale near zero.

In other words, the above result can be written as $(-1)^0 \times 1.001_{(2)} \times 2^2$ which yields the integer components as s = 0, b = 2, significant (m) = 1.001, mantissa = 001 and e = 2. The corresponding single precision floating number can be represented in binary as shown below,

S	E	M
1	10000001	00100000000000000000000

Where the exponent field is supposed to be 2, yet encoded as 129 (127+2)
called ***biased exponent***. The exponent field is in plain binary format which also represents negative exponents with an encoding (like sign magnitude, 1's complement, 2's complement, etc.). The biased exponent is used for the representation of negative exponents. The biased exponent has advantages over other negative representations in performing bitwise comparing of two floating point numbers for equality.
A ***bias*** of $(2^{n-1} - 1)$, where **n** is # of bits used in exponent, is added to the exponent (e) to get biased exponent (*E*). So, the biased exponent (*E*) of *single precision* number can be obtained as

$$E = e + 127$$

The range of exponent in single precision format is -128 to +127. Other values are used for special symbols.

Note: When we unpack a floating point number the exponent obtained is the biased exponent. Subtracting 127 from the biased exponent we can extract unbiased exponent.

Double Precision Format:

As mentioned in Table – 1 the double precision format has 52 bits for significant (1 represents implied bit), 11 bits for exponent and 1 bit for sign. All other definitions are same for double precision format, except for the size of various components.

Precision:
The smallest change that can be represented in floating point representation is called as precision. The fractional part of a single precision normalized number has exactly 23 bits of resolution, (24 bits with the implied bit). This corresponds to $\log_{(10)} (2^{23}) = 6.924 = 7$ (the characteristic of logarithm) decimal digits of accuracy. Similarly, in case of double precision numbers the precision is $\log_{(10)} (2^{52}) = 15.654 = 16$ decimal digits.

Accuracy:
Accuracy in floating point representation is governed by number of significant bits, whereas range is limited by exponent. Not all real numbers can exactly be represented in floating point format. For any numberwhich is not floating point number, there are two options for floating point approximation, say, the closest floating point number *less than x* as x_ and the closest floating point number *greater than x* as x+. A ***rounding*** operation is performed on number of significant bits in the mantissa field based on the selected mode. The ***round down*** mode causes x set to x_, the ***round up*** mode causes x set to x+, the ***round towards zero*** mode causes x is either x_ or x+ whichever is between zero and. The ***round to nearest*** mode sets x to x_ or x+ whichever is nearest to x. Usually ***round to nearest*** is most used mode. The closeness of floating point representation to the actual value is called as ***accuracy***.

Special Bit Patterns:
The standard defines few special floating point bit patterns. Zero can't have most significant 1 bit, hence can't be normalized. The hidden bit representation requires a special technique for storing zero. We will have two different bit patterns +0 and -0 for the same numerical value zero. For single precision floating point representation, these patterns are given below,

0 00000000 00000000000000000000000 = +0

1 00000000 00000000000000000000000 = -0

Similarly, the standard represents two different bit patters for +INF and -INF. The same are given below,

0 11111111 00000000000000000000000 = +INF

1 11111111 00000000000000000000000 = -INF

All of these special numbers, as well as other special numbers (below) are subnormal numbers, represented through the use of a special bit pattern in the exponent field. This slightly reduces the exponent range, but this is quite acceptable since the range is so large.

An attempt to compute expressions like 0 x INF, 0 ÷ INF, etc. make no mathematical sense. The standard calls the result of such expressions as Not a Number (NaN). Any subsequent expression with NaN yields NaN. The representation of NaN has non-zero significant and all 1s in the exponent field. These are shown below for single precision format (x is don't care bits),

x 11111111 1**m**00000000000000000000000
Where ***m*** can be 0 or 1. This gives us two different representations of NaN.

0 11111111 11000000000000000000000 _____ Signaling NaN (SNaN)

0 11111111 10000000000000000000000 _____Quiet NaN (QNaN)

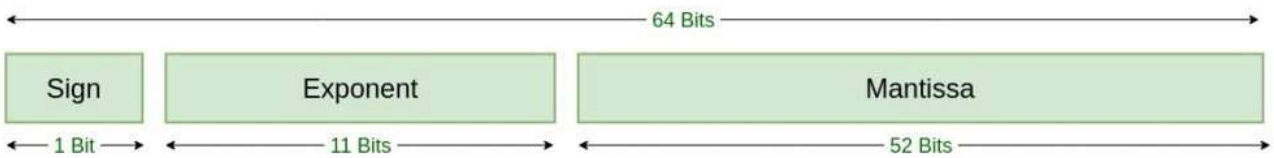
Usually QNaN and SNaN are used for error handling. QNaN do not raise any exceptions as they propagate through most operations. Whereas SNaN are which when consumed by most operations will raise an invalid exception.

IEEE Standard 754 Floating Point Numbers

The IEEE Standard for Floating-Point Arithmetic (IEEE 754) is a technical standard for floating-point computation which was established in 1985 by the **Institute of Electrical and Electronics Engineers (IEEE)**. The standard addressed many problems found in the diverse floating point implementations that made them difficult to use reliably and reduced their portability. IEEE Standard 754 floating point is the most common representation today for real numbers on computers, including Intel-based PC's, Macs, and most Unix platforms.
There are several ways to represent floating point number but IEEE 754 is the most efficient in most cases. IEEE 754 has 3 basic components:

1. **The Sign of Mantissa –**
   This is as simple as the name. 0 represents a positive number while 1 represents a negative number.
2. **The Biased exponent –**
   The exponent field needs to represent both positive and negative exponents. A bias is added to the actual exponent in order to get the stored exponent.
3. **The Normalised Mantissa –**
   The mantissa is part of a number in scientific notation or a floating-point number, consisting of its significant digits. Here we have only 2 digits, i.e. O and 1. So a normalised mantissa is one with only one 1 to the left of the decimal.

IEEE 754 numbers are divided into two based on the above three components: single precision and double precision.



Double Precision
IEEE 754 Floating-Point Standard

TYPES	SIGN	BIASED EXPONENT	NORMALISED MANTISA	BIAS
Single precision	1(31st bit)	8(30-23)	23(22-0)	127
Double precision	1(63rd bit)	11(62-52)	52(51-0)	1023

Example –

```
85.125

85 = 1010101

0.125 = 001

85.125 = 1010101.001

       =1.010101001 x 2^6

sign = 0


1. Single precision:

biased exponent 127+6=133

133 = 10000101

Normalised mantisa = 010101001

we will add 0's to complete the 23 bits


The IEEE 754 Single precision is:

= 0 10000101 01010100100000000000000

This can be written in hexadecimal form 42AA4000

2. Double precision:
biased exponent 1023+6=1029
1029 = 10000000101
Normalised mantisa = 010101001
we will add 0's to complete the 52 bits

The IEEE 754 Double precision is:
= 0 10000000101 0101010010000000000000000000000000000000000000000000
This can be written in hexadecimal form 4055480000000000
```

Special Values: IEEE has reserved some values that can ambiguity.

- **Zero –**
  Zero is a special value denoted with an exponent and mantissa of 0. -0 and +0 are distinct values, though they both are equal.
- **Denormalised –**
  If the exponent is all zeros, but the mantissa is not then the value is a denormalized number. This means this number does not have an assumed leading one before the binary point.
- **Infinity –**
  The values +infinity and -infinity are denoted with an exponent of all ones and a mantissa of all zeros. The sign bit distinguishes between negative infinity and positive infinity. Operations with infinite values are well defined in IEEE.
- **Not A Number (NAN) –**
  The value NAN is used to represent a value that is an error. This is represented when exponent field is all ones with a zero sign bit or a mantissa that it not 1 followed by zeros. This is a special value that might be used to denote a variable that doesn’t yet hold a value.

EXPONENT	MANTISA	VALUE
0	0	exact 0
255	0	Infinity
0	not 0	denormalised
255	not 0	Not a number (NAN)